Module 5: Developing Flexible NoSQL Solutions

AWS Academy Cloud Developing

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 1: Introduction

Module 5: Developing Flexible NoSQL Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module objectives

At the end of this module, you should be able to do the following:

- Recognize different database options on Amazon Web Services (AWS)
- Identify the features of Amazon DynamoDB
- Describe the components of DynamoDB
- Explain how DynamoDB uses partitions
- Indicate how indexes are used with DynamoDB
- Describe how DynamoDB keeps data consistent
- Recognize when streaming and global tables are used
- Explain the backup and restore process
- Develop with DynamoDB by using the AWS software development kits (SDK)

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module overview

Sections

1. Introduction
2. Introducing AWS database options
3. Key concepts for DynamoDB
4. Partitions and data distribution
5. Secondary indexes
6. Read/write throughput
7. Streams and global tables
8. Backup and restore
9. Basic operations for DynamoDB tables

Lab

Working with Amazon DynamoDB

Activity

Calculating RCUs and WCUs

Knowledge check

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

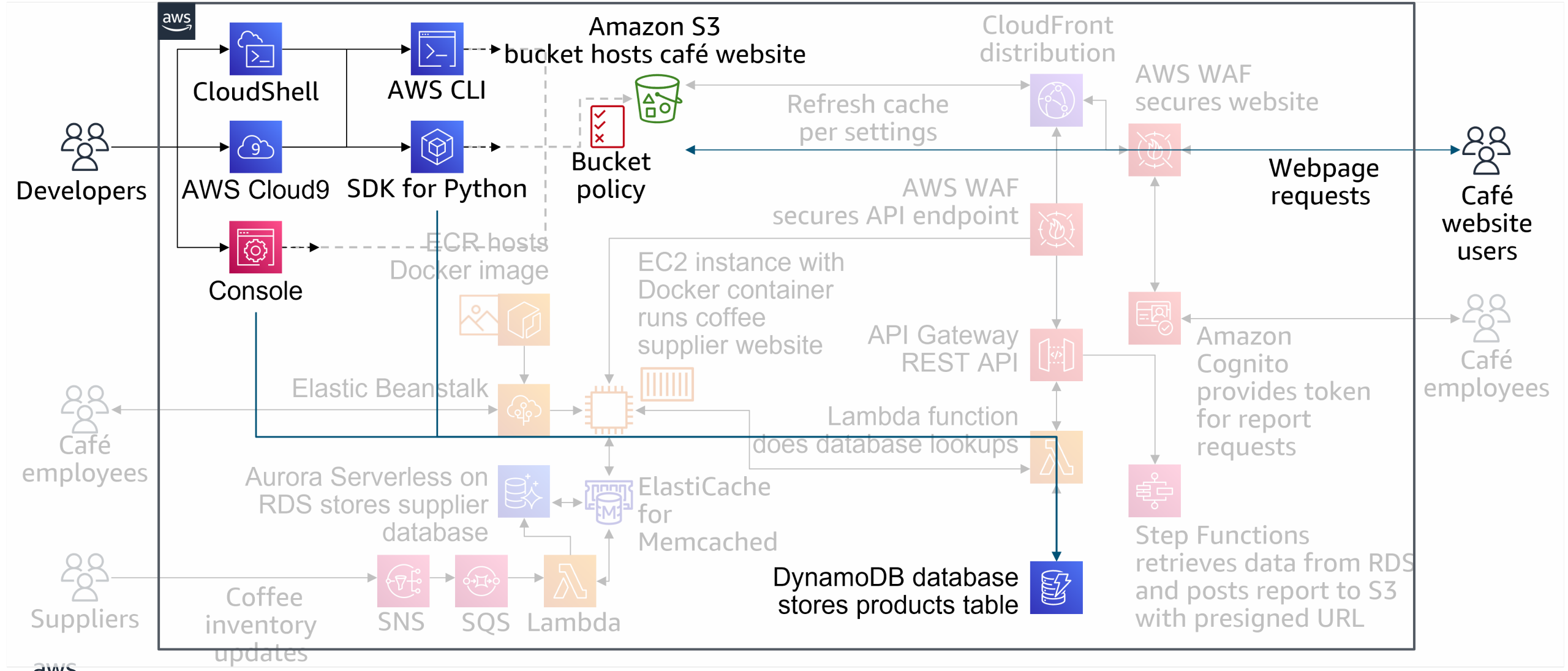
Café business requirement

The café has a basic website now. Sofía wants to modify the site so that information can be updated dynamically and staff members can retrieve information about the product inventory.

```
cursor.close()

sql = "INSERT INTO hive (dev_id, humidity, temperature) VALUES (%s, %s, %s)"

def on_connect(client, userdata, flags, rc):
    print("Connected with result code " + str(rc))

    client.subscribe("+/devices/+/up")

def on_message(client, userdata, msg):
    print(msg.topic + " &&& " + str(msg.payload)+"\n")
    data = json.loads(msg.payload.decode('utf8').replace("'", '"'))
    val = (data["dev_id"], data["payload_fields"]["humidity"], data["payload_fields"]["temperature"])
    connection = mysql.connector.connect(
        host="bees-mariaDB",
        user="bees",
        passwd=mysql_password,
        database="bees"
    )
    cursor = connection.cursor()
    cursor.execute(sql, val)
    connection.commit()
    print(mycursor.rowcount, "record inserted.")
    cursor.close()
    connection.close()

client = mqtt.Client()
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

NoSQL databases as part of developing a cloud application



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 2: Introducing AWS database options

Module 5: Developing Flexible NoSQL Solutions

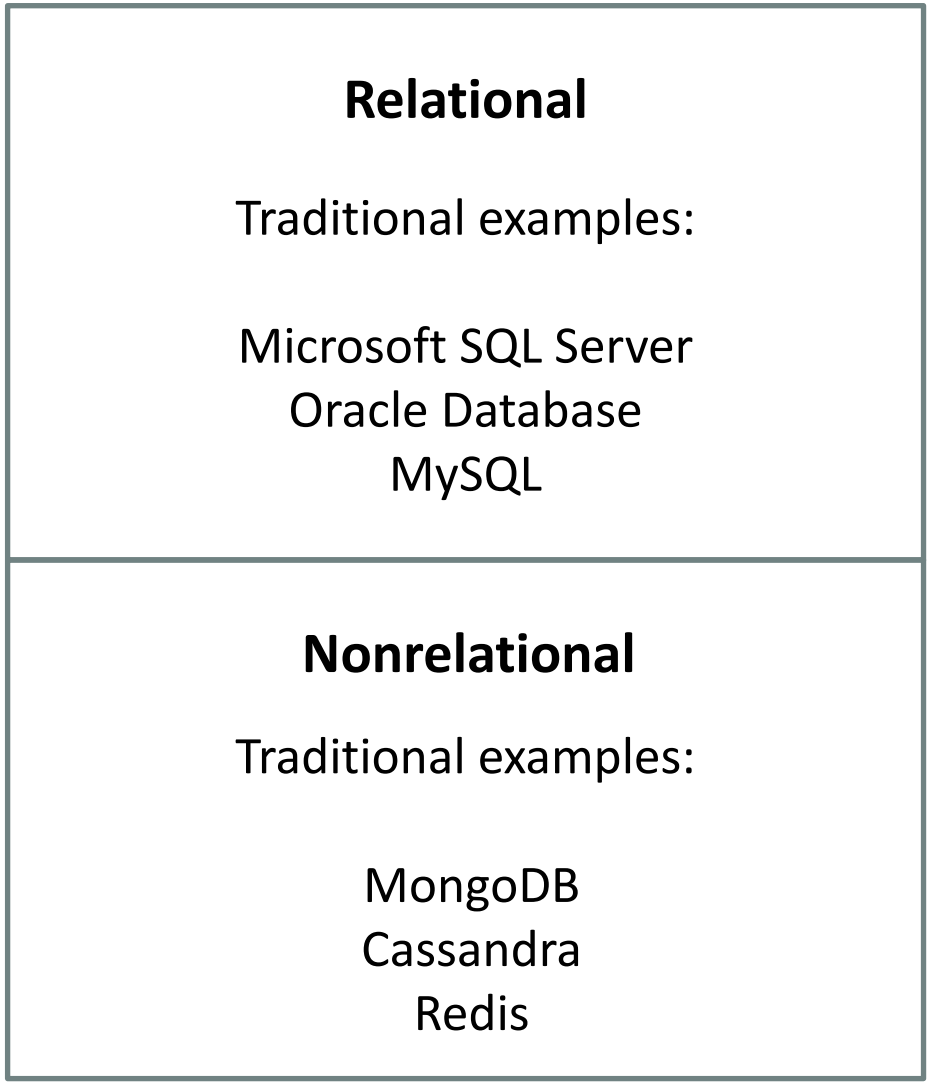
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Database types

Databases (DBs) are excellent solutions for persistent storage.

The two major types of databases are:

- Relational databases
- Nonrelational databases



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Database type: Relational

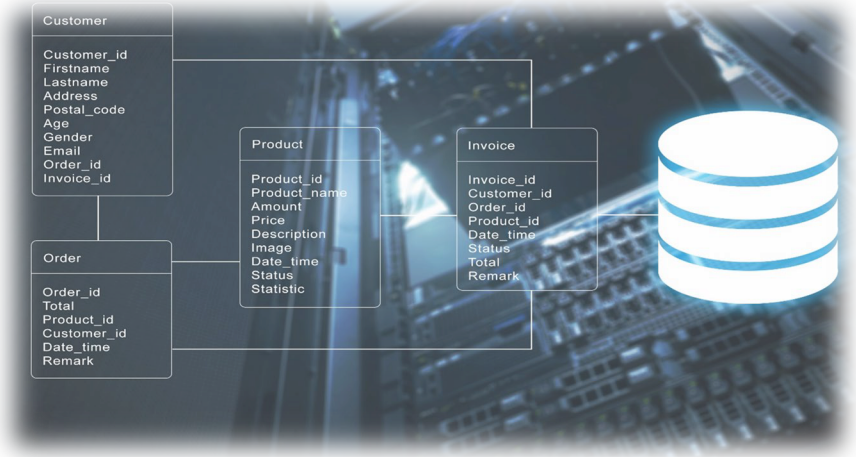
When to choose a relational database:

- You must use strict schema rules and must enforce data quality
- Your database doesn't need extreme read/write capacity
- If you have a relational dataset that doesn't require extreme performance, a relational database management solution (RDBMS) can be the lowest-effort solution.

Relational

Traditional examples:

Microsoft SQL Server
Oracle Database
MySQL



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Database type: Nonrelational

When to choose a nonrelational database:

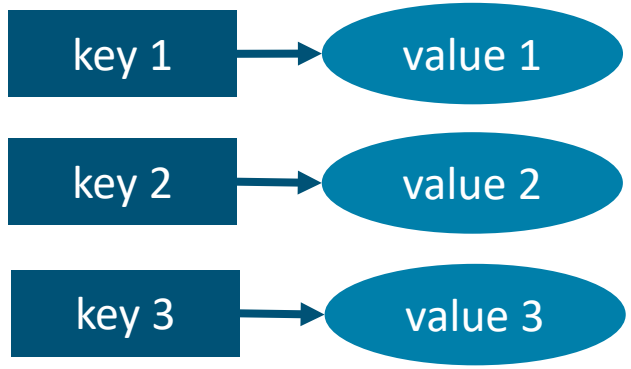
- You need your database to scale horizontally
- Your data doesn't lend itself well to traditional schemas (nonrelational databases can use key-value pairs)
- Your read/write rates exceed rates that can be economically supported through traditional structured query language (SQL) databases

Nonrelational

Traditional examples:

MongoDB
Cassandra
Redis

Key-value



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Comparing structured data storage

Feature	Relational or SQL	Nonrelational or NoSQL
Data Storage	Rows and columns	Key-value, documents, and graphs
Schemas	Fixed	Dynamic
Querying	SQL-based queries	Focused on collection of documents
Scalability	Vertical	Horizontal

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

AWS database options: Relational

Amazon Relational Database Service (Amazon RDS)

Amazon Aurora

AWS offers more databases—these services are commonly used examples.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

AWS database options: Nonrelational

Amazon DynamoDB

Amazon ElastiCache

Amazon Neptune

AWS offers more databases—these services are commonly used examples.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 2 key takeaways

- Relational databases:
  - Strict schema
  - Doesn’t need extreme performance
- Nonrelational databases:
  - Horizontal scaling
  - Non-traditional schema
- AWS offers several different services to support both relational and nonrelational databases.

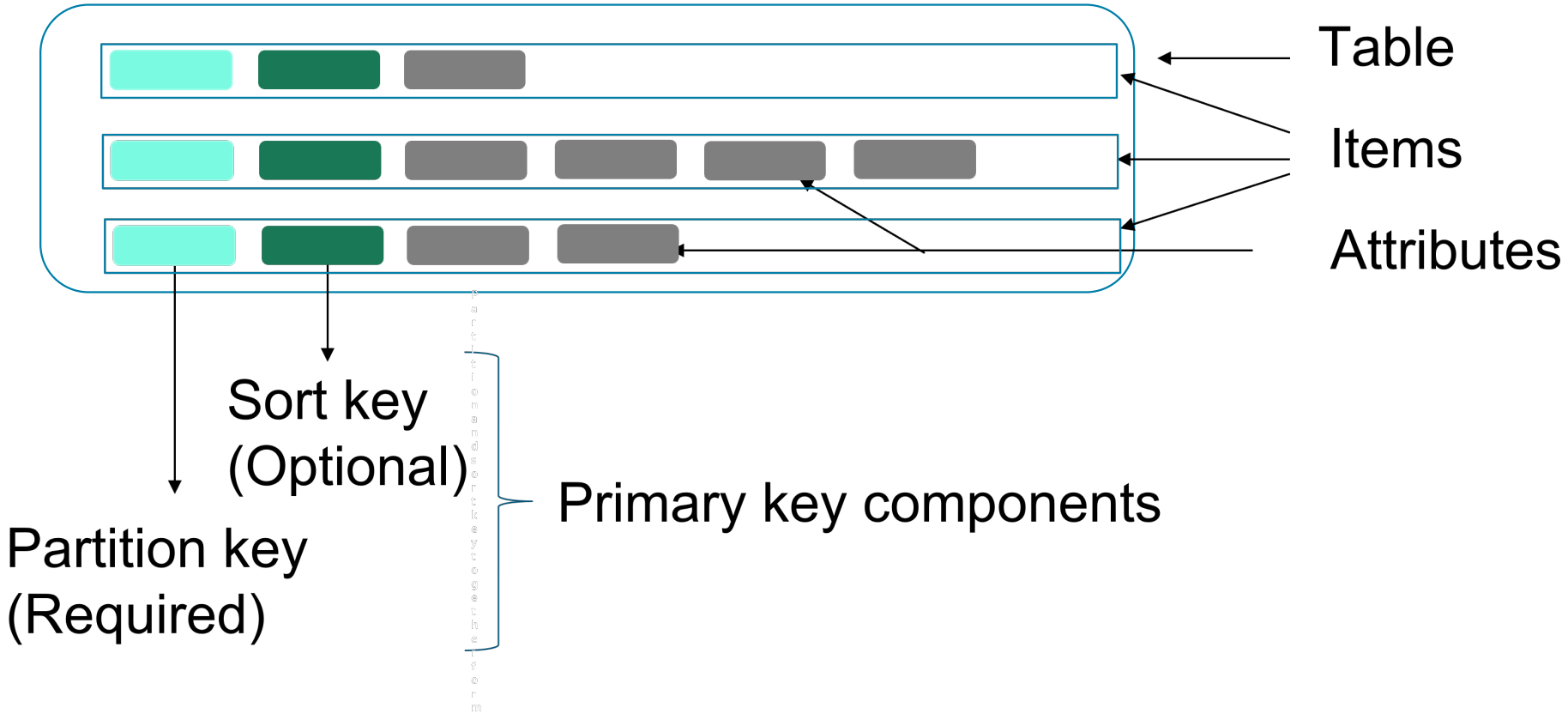
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 3: Key concepts for DynamoDB

Module 5: Developing Flexible NoSQL Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Basic components of DynamoDB



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Types of primary keys

Simple primary key
(partition key only)

SensorId (Partition Key)	Latitude	Longitude
SensorA	40.712784	-74.005941
SensorB	35.689488	139.691706

SensorLocation table

Composite primary key
(partition key and sort key)

SensorId (Partition Key)	Time (Sort Key)	Value
SensorA	2021-01-03T10:15:30	30
SensorA	2021-01-04T10:19:30	35
SensorB	2021-03-04T11:21:20	28

SensorReadings table

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Items and attribute types

Key-value item

ItemNumber	Title	LeadActor	Price	InCirculation
12	"My Favorite Video"	"Mateo Jackson"	5.00	true

String

Number

Boolean

Document item

JSON

```
{
  "person_id" : 123,
  "last_name" : "Doe",
  "first_name" : "John",
  "next_anniversary" :
  {
    "year" : 2021,
    "month" : 5,
    "day" : 30
  },
  "children"  :
    ["Paulo", "Richard", "Jane", "Mary"
]
}
```

Map

List

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 3 key takeaways

- Components of DynamoDB:
  - Table
  - Items
  - Attributes
- Types of primary keys:
  - Simple primary key
  - Composite primary key
- An item has attributes.
- Different items in the same table can have a different number of attributes.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 4: Partitions and data distribution

Module 5: Developing Flexible NoSQL Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Partitions

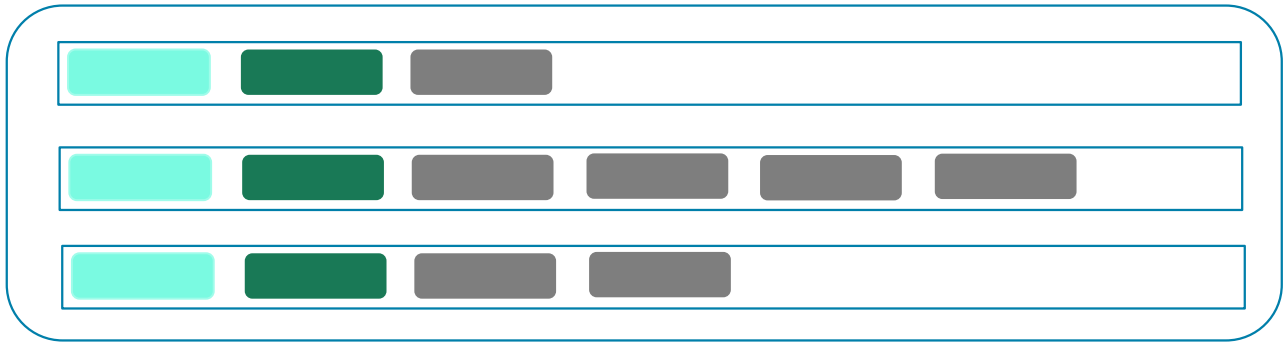
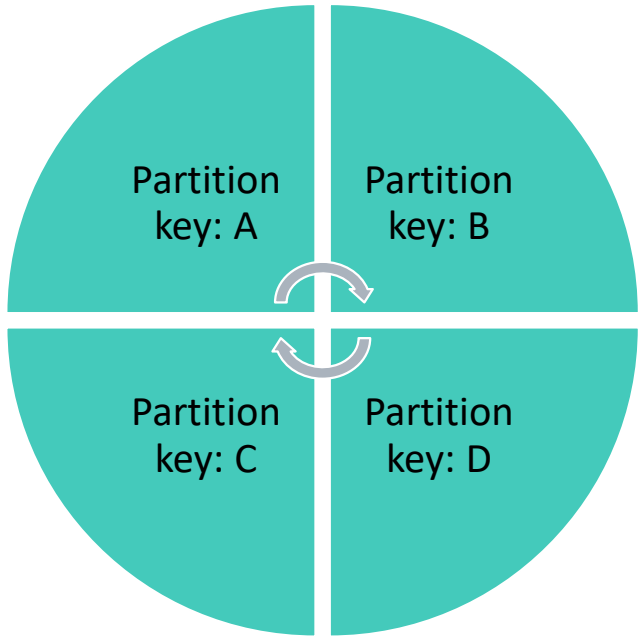
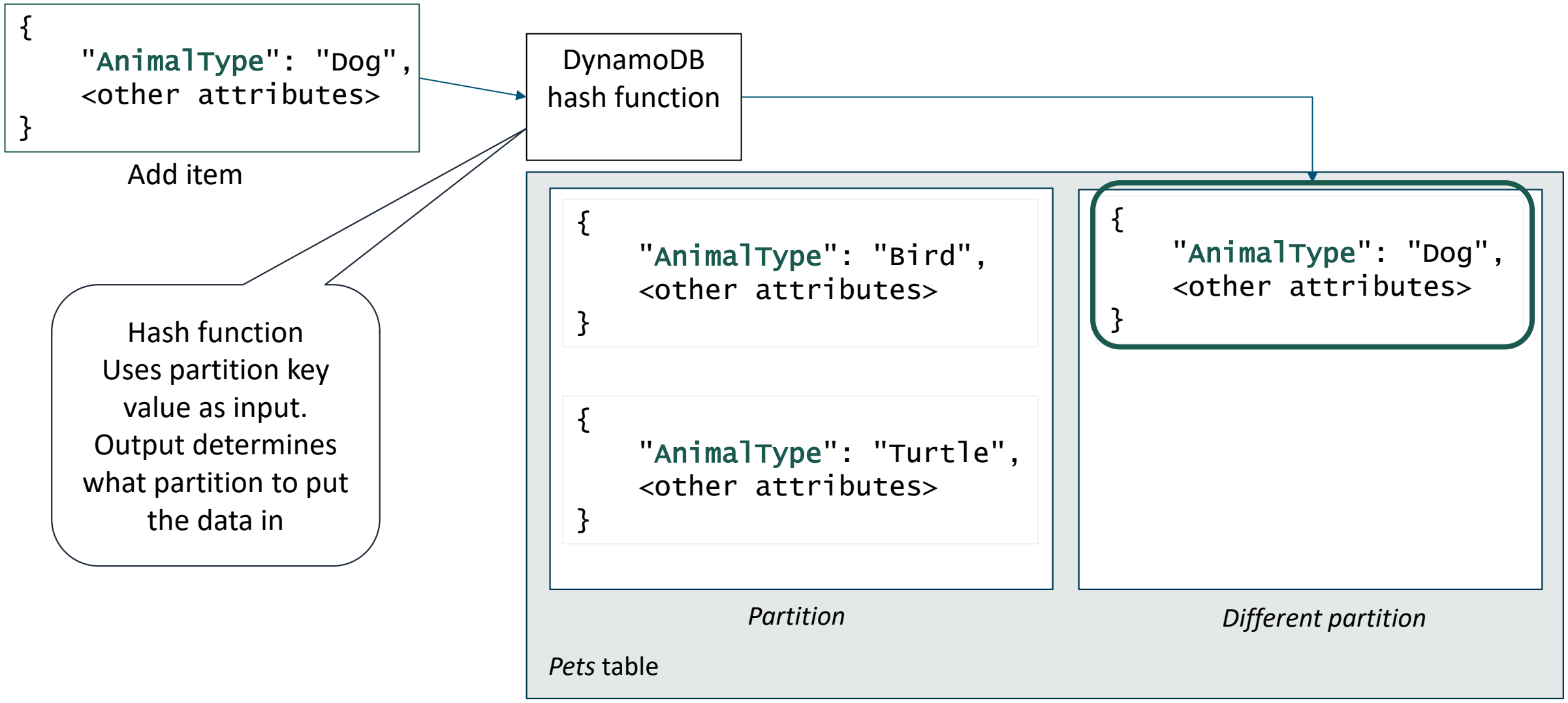


Table data is stored in partitions based on the partition key.



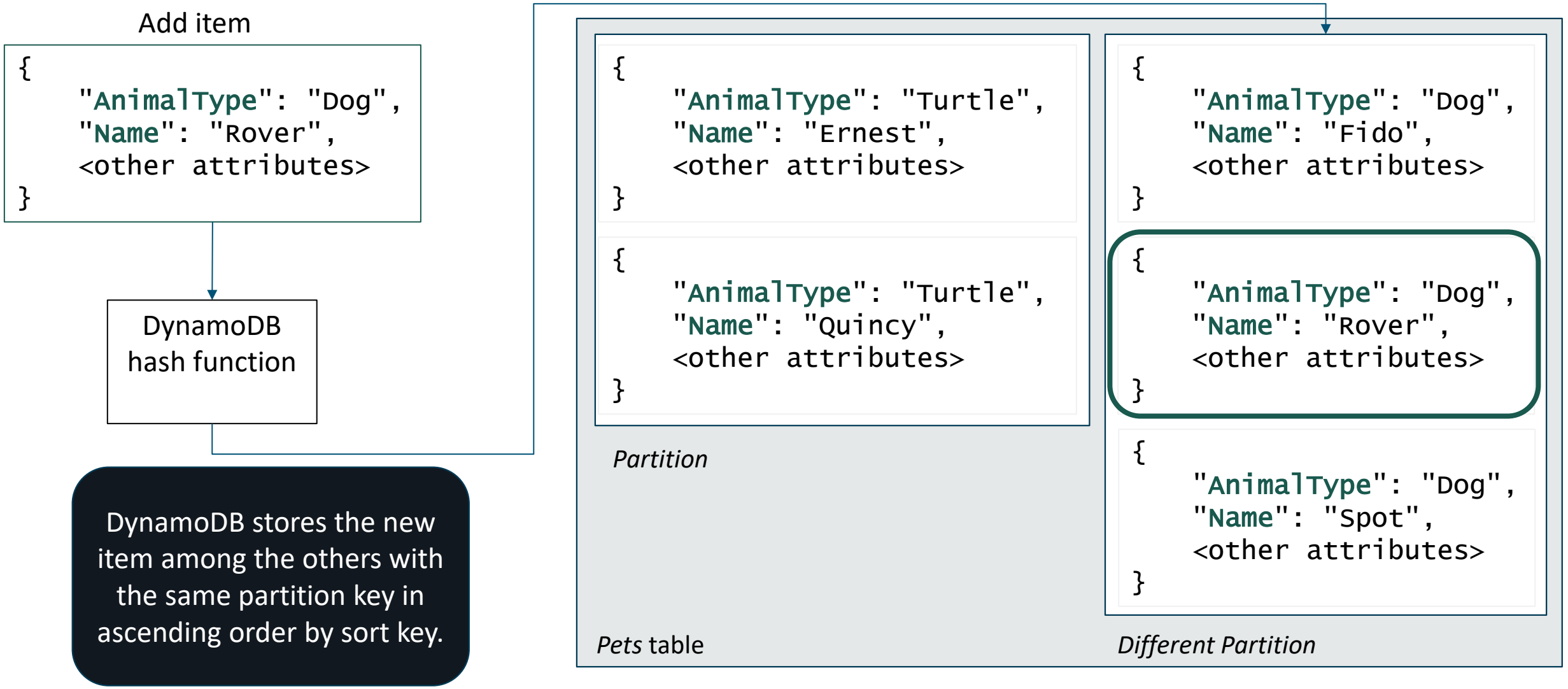
aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Partitioning with a simple primary key: Example



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Partitioning with a composite primary key: Example



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 4 key takeaways

- DynamoDB stores data in partitions based on the partition key.
- With a simple primary key, the key determines which partitions are selected.
- With a composite primary key, the key still determines which partitions are selected on the key, but the order is based on the sort key.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 5: Secondary indexes

Module 5: Developing Flexible NoSQL Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Secondary indexes

- Use to query data based on non-primary-key attributes
  - Secondary index defines an alternate key
- Contain:
  - Alternate-key attributes
  - Primary-key attributes
  - Optional subset of other attributes from the base table (projected attributes)
- Can be one of two types:
  - Global secondary index
  - Local secondary index

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Global secondary index example

Music Table

```
{
    "Artist": "No One You Know",
    "SongTitle": "My Dog Spot",
    "AlbumTitle": "Hey Now",
    "Price": 1.98,
    "Genre": "Country",
    "CriticRating": 8.4
}
{
    "Artist": "No One You Know",
    "SongTitle": "Somewhere Down the Road",
    "AlbumTitle": "Somewhat Famous",
    "Genre": "Country",
    "CriticRating": 8.4
    "Year": 1984
}
{
    "Artist": "The Acme Band",
    "SongTitle": "Look Out, World",
    "AlbumTitle": "The Buck Starts Here",
    "Price": 0.99
    "Genre": "Rock"
}
```

GenreAlbumTitle Secondary Index

```
{
    "Genre": "Country",
    "AlbumTitle": "Hey Now",
    "Artist": "No One You Know",
    "SongTitle": "My Dog Spot",
}
{
    "Genre": "Country",
    "AlbumTitle": "Somewhat Famous",
    "Artist": "No One You Know",
    "SongTitle": "Somewhere Down the Road",
}
{
    "Genre": "Rock"
    "AlbumTitle": "The Buck Starts Here",
    "Artist": "The Acme Band",
    "SongTitle": "Look Out, World",
}
```

Different partition key and sort key from the keys in the table

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Local secondary index example

Music Table

```
{
    "Artist": "No One You Know",
    "SongTitle": "My Dog Spot",
    "AlbumTitle": "Hey Now",
    "Price": 1.98,
    "Genre": "Country",
    "CriticRating": 8.4
}
{
    "Artist": "No One You Know",
    "SongTitle": "Somewhere Down the Road",
    "AlbumTitle": "Somewhat Famous",
    "Genre": "Country",
    "CriticRating": 8.4
    "Year": 1984
}
{
    "Artist": "The Acme Band",
    "SongTitle": "Look Out, World",
    "AlbumTitle": "The Buck Starts Here",
    "Price": 0.99
    "Genre": "Rock"
}
```

ArtistAlbumTitle Local Secondary Index

```
{
    "Artist": "No One You Know",
    "AlbumTitle": "Hey Now",
    "SongTitle": "My Dog Spot",
    "Genre": "Country",
}
{
    "Artist": "No One You Know",
    "AlbumTitle": "Somewhat Famous",
    "SongTitle": "Somewhere Down the Road",
    "Genre": "Country",
}
{
    "Artist": "The Acme Band"
    "AlbumTitle": "The Buck Starts Here",
    "SongTitle": "Look Out, World",
    "Genre": "Rock",
}
```

Same partition key as the table but different sort key

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 5 key takeaways

- Secondary indexes contain a subset of attributes and an alternate key or keys to support query operations.
- Global secondary indexes have different partition keys and sort keys from the keys in the base table.
- Local secondary indexes have the same partition key as the base table, but a different sort key.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 6: Read/write throughput

Module 5: Developing Flexible NoSQL Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Read consistency

- DynamoDB maintains multiple copies of data for durability.
  - All copies of your data are usually consistent within a second after a write operation.
- Eventually consistent read might return slightly stale data if a read operation is performed immediately after a write operation.
  - Use case: Blog posts.
- Strongly consistent read returns the most up-to-date data.
  - Use cases: Scoreboards, booking systems, financial algorithms.
- You can specify the preferred consistency level for reading data.

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

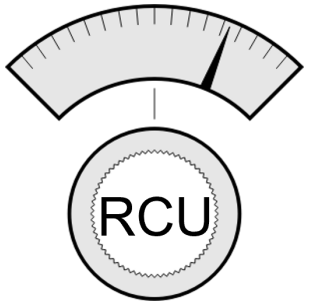
Transactions

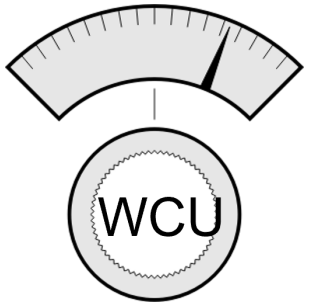
- Simplify the developer experience of making coordinated, all-or-nothing changes (inserts, deletes, or updates) to multiple items both within and across tables
- Provide atomicity, consistency, isolation, and durability (ACID), which help you maintain data correctness in your applications
- Use cases
  - Processing financial transactions
  - Fulfilling and managing orders
  - Building multiplayer game engines
  - Coordinating actions across distributed components and services

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Provisioned read/write throughput

- Throughput = Maximum amount of capacity that an application can consume from a table or index
- It is divided evenly among partitions
- Read capacity unit (RCU)
  - Number of strongly consistent reads per second of items that are up to 4 KB in size
  - Eventually consistent reads use half of the provisioned read capacity
- Write capacity unit (WCU)
  - Number of 1-KB writes per second





aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

On-demand read/write throughput

- Pay-per-request pricing for read and write requests
- DynamoDB adapts rapidly to accommodate the workload
- Request rate is limited by throughput default table limits
- Create or update a table to use on-demand mode
- Use cases
  - You create new tables with unknown workloads
  - You have unpredictable application traffic
  - You prefer the ease of paying for only what you use

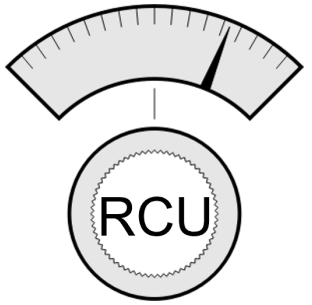
aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Activity: Calculating RCUs and WCUs

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Activity: Calculating RCU

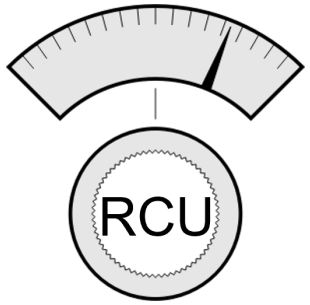
- Suppose that you must read 20 items that are 11 KB in size every second, with eventual consistency.
- How many RCUs must you provision?
- 1 RCU = 1 strongly consistent, or 2 eventually consistent, reads per second for items up to 4 KB in size



aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Calculating RCU: Answer

Answer: 30 RCUs

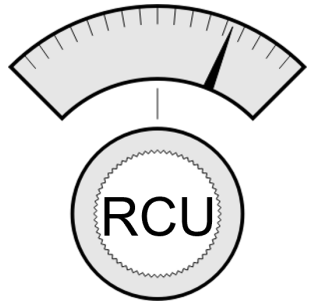


- Round 11 KB up to the next multiple of 4: *12*
- Divide by 4 KB per RCU: *3*
- Multiply by items read per second: *60*
- Divide by 2 for eventual consistency: *30*

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Activity: Calculating WCU

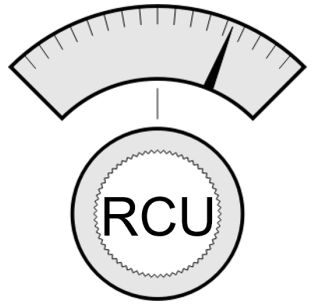
- Suppose that you must write 120 items that are 7 KB in size every minute.
- How many WCUs must you provision?
- 1 WCU = 1 write per second, for items up to 1 KB in size



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Calculating WCU: Answer

Answer: 14 WCUs



- WCUs required to write each 7-KB item: *7*
- Number of items per second: *2*
- Multiply WCU per item by items per second: *14*

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 6 key takeaways

- DynamoDB provides two read consistencies:
  - Eventual consistency: Data might be stale
  - Strong consistency: Up-to-date data
- DynamoDB transactions offer insert, delete, and update operations that are all-or-nothing and ACID compliant.
- Read and write throughput can be either of the following types:
  - Provisioned (set throughput as RCU/WCU)
  - On-demand (changes with demand)

aws

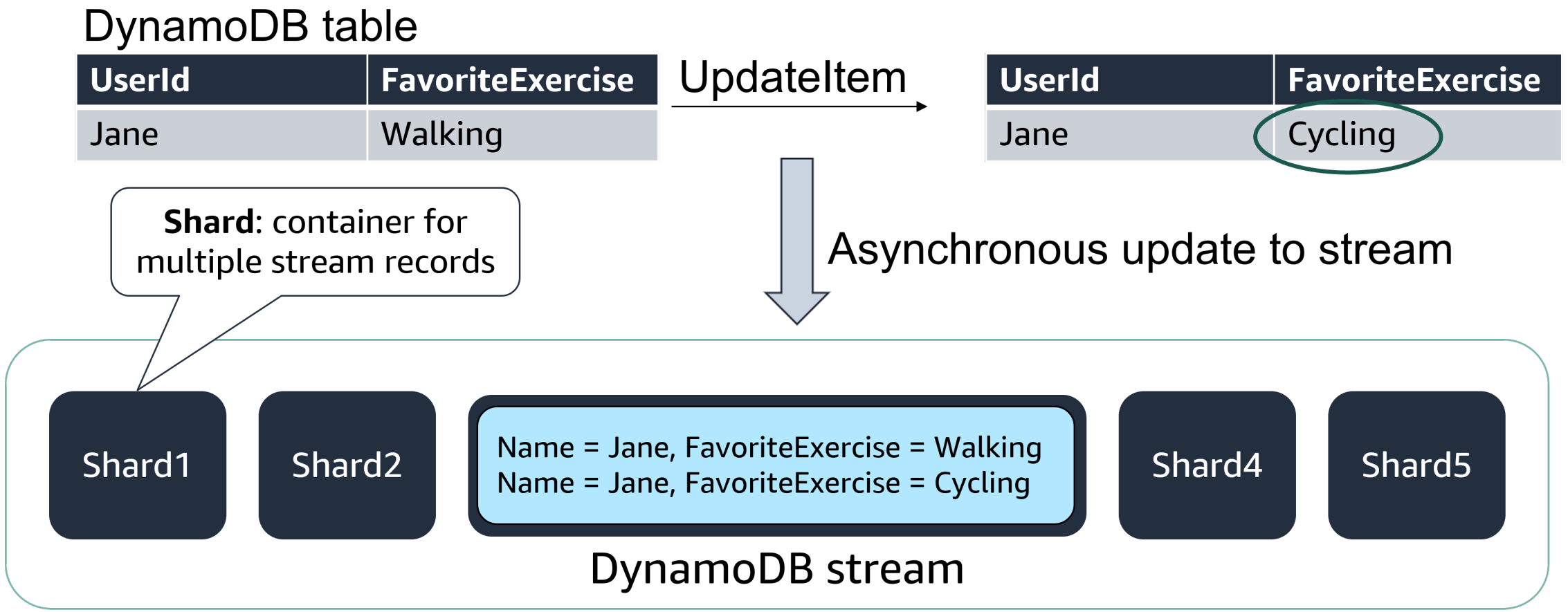
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 7: Streams and global tables

Module 5: Developing Flexible NoSQL Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

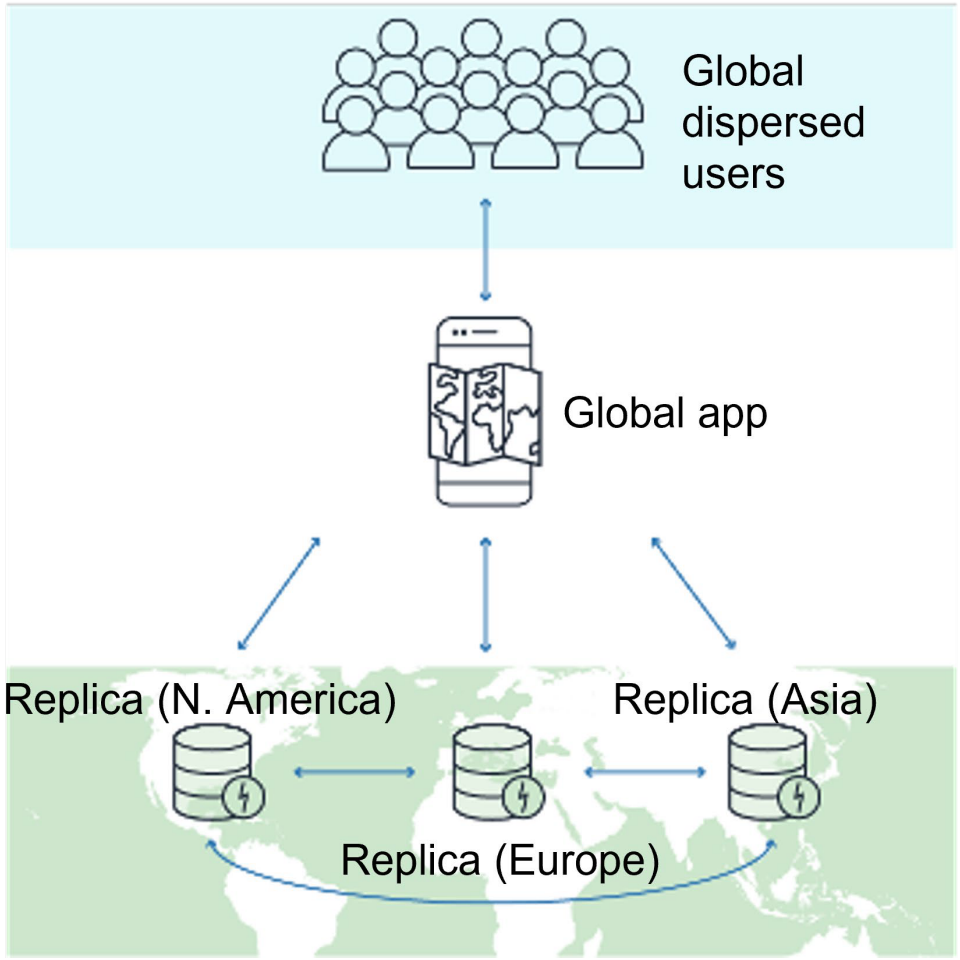
DynamoDB Streams



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

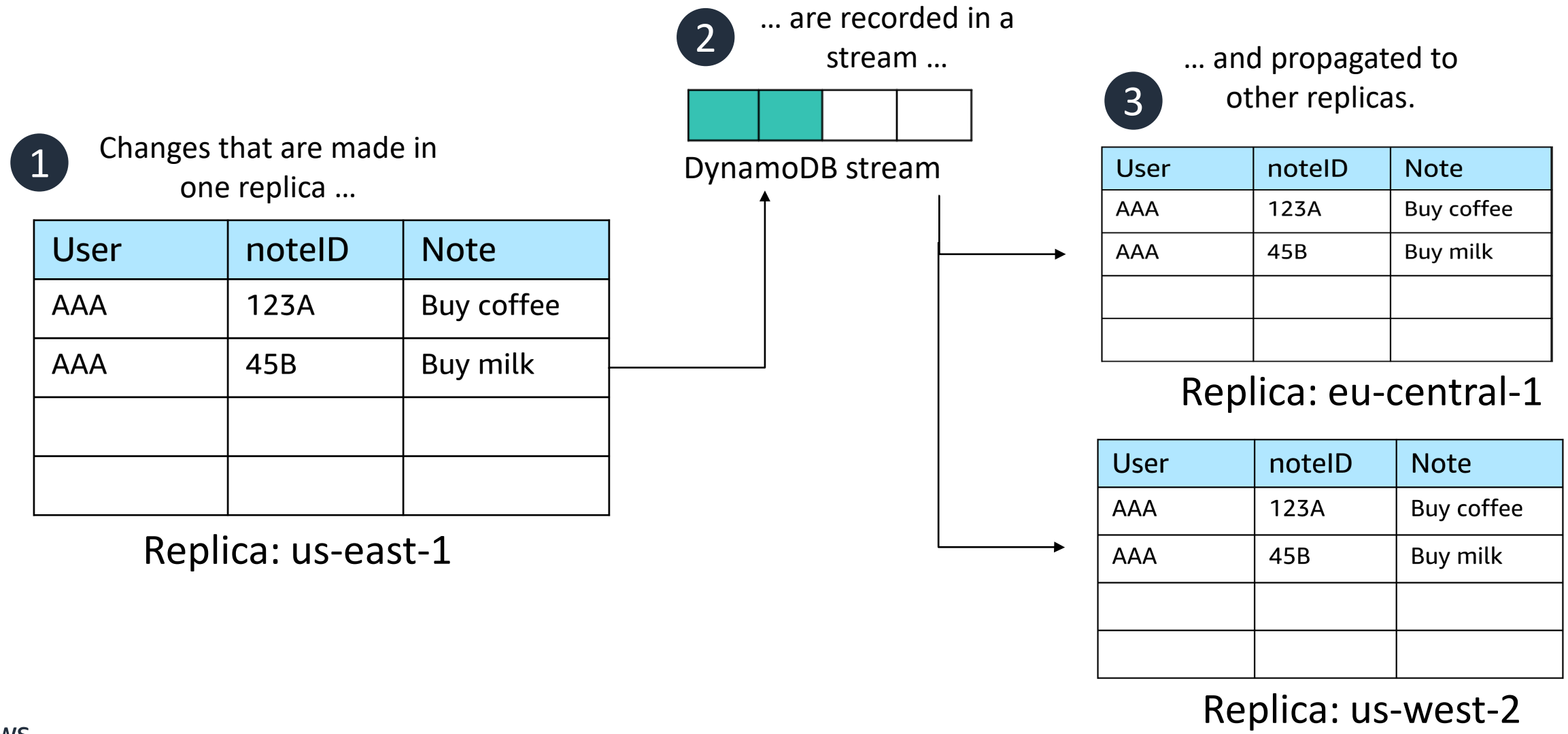
Global tables

- Global tables offer a solution to deploy a multi-Region, multi-active database without needing to build and maintain your own replication solution.
- You specify the AWS Regions where you want the table to be available.



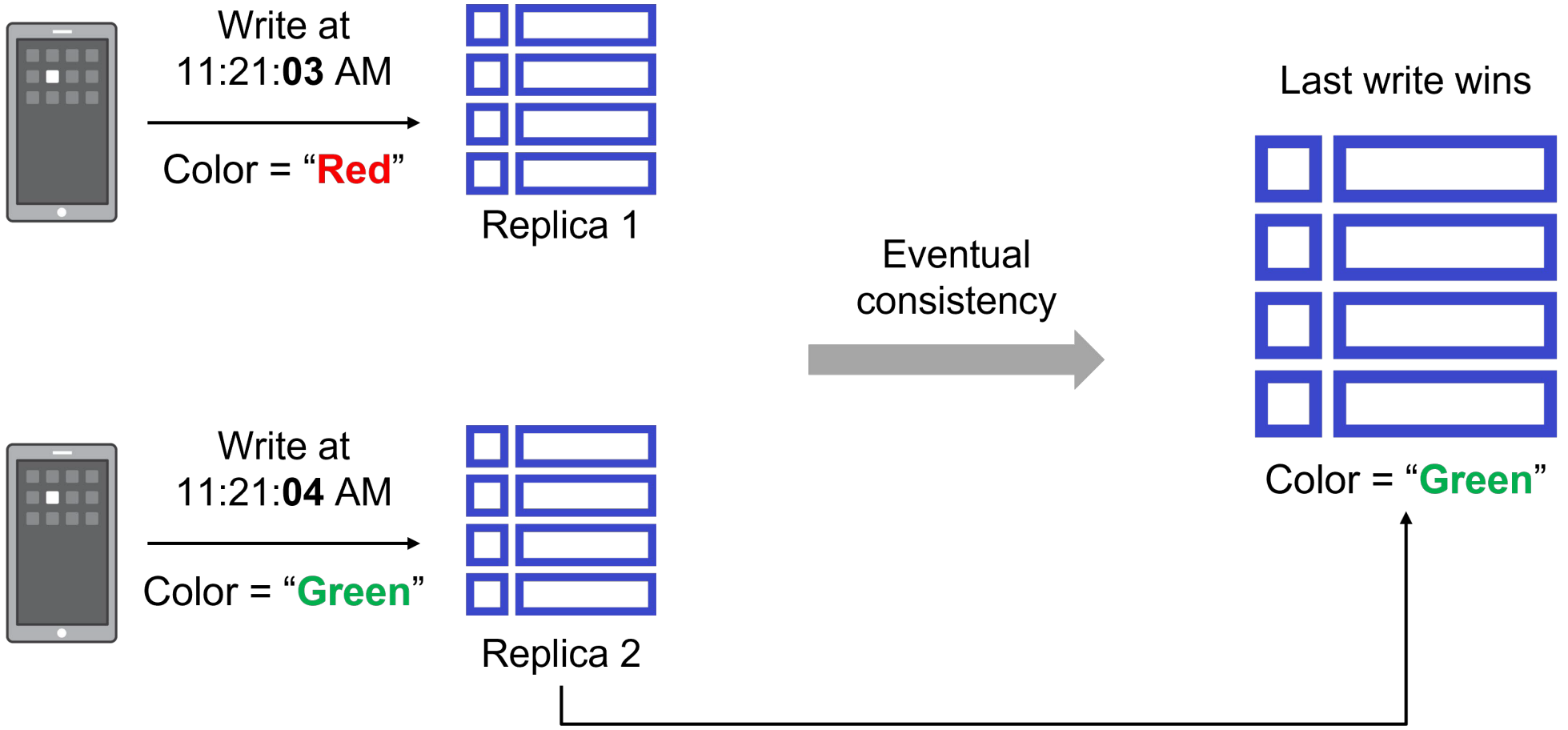
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Global tables: Data replication



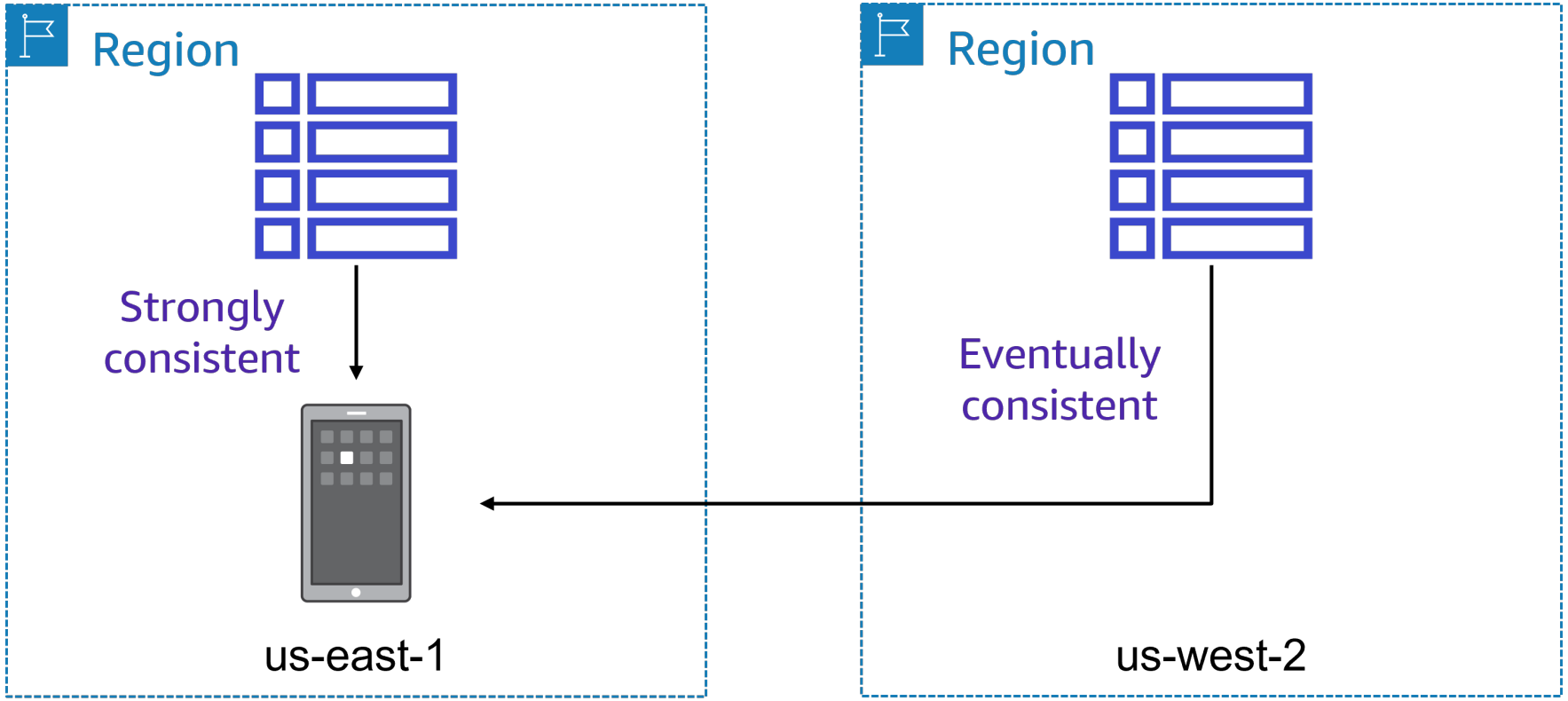
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Global tables: Concurrent updates



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Global tables: Read consistency



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 7 key takeaways

- DynamoDB streams are observed changes in data.
- Streams are composed of shards of data.
- Streams can be consumed by other AWS services, like AWS Lambda.
- Global tables replicate DynamoDB tables across AWS Regions.

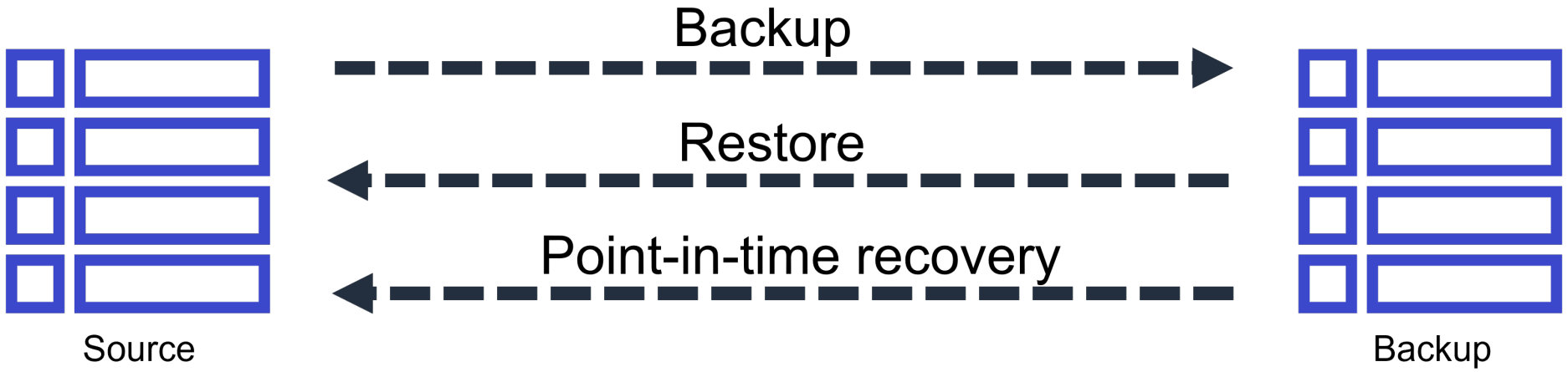
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 8: Backup and restore

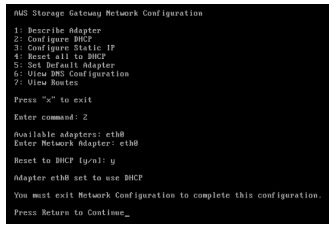
Module 5: Developing Flexible NoSQL Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

On-demand backups and point-in-time recovery



AWS Management Console



AWS Command Line Interface (AWS CLI)

Amazon DynamoDB API

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 9: Basic operations for DynamoDB tables

Module 5: Developing Flexible NoSQL Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Amazon DynamoDB API operations

- Control operations: Create and manage DynamoDB tables
  - For example, CreateTable
- Data operations: Create, read, update, and delete actions on data in a table
  - Example: Write operations – PutItem, UpdateItem, DeleteItem,
  - Example: Read operations – GetItem, Query, Scan
- Batch operations: Get and write batches of items in a DynamoDB table
- Transaction operations: Make coordinated, all-or-nothing changes to multiple items both within and across tables

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Creating a table

TableName is the name of the table to create.

KeySchema specifies the attributes that make up the primary key for the table.

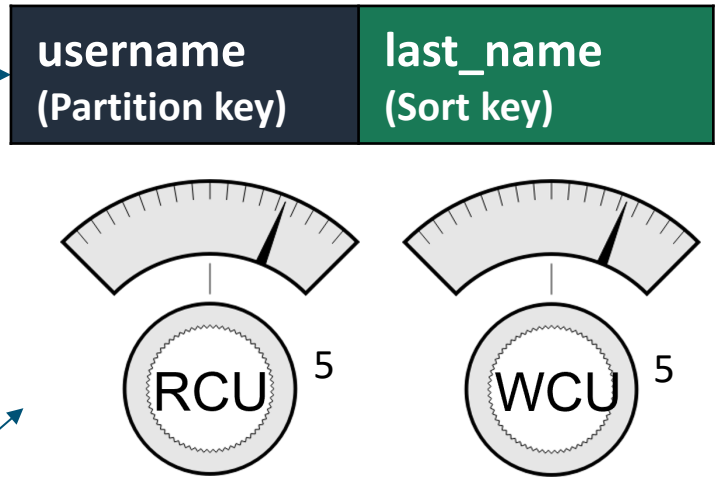
AttributeDefinitions is an array of attributes that describes the key schema for the table.

ProvisionedThroughput represents the provisioned throughput settings for the table.

```
table = dynamodb.create_table(
    TableName = 'users',
    KeySchema = [
        {
            'AttributeName': 'username',
            'KeyType': 'HASH'
        },
        {
            'AttributeName': 'last_name',
            'KeyType': 'RANGE'
        }
    ],
    AttributeDefinitions = [
        {
            'AttributeName': 'username',
            'AttributeType': 'S'
        },
        {
            'AttributeName': 'last_name',
            'AttributeType': 'S'
        }
    ],
    ProvisionedThroughput = {
    'ReadCapacityUnits': 5,
    'WriteCapacityUnits': 5
    }
)
```

Create_table:
Creates a new table

users Table



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Creating an item

Put_item: Creates a new item or replaces (overwrites) an existing item

users Table: Before

username (Partition key)	last_name (Sort key)

```
table.put_item(
    Item = {
        'username': 'janedoe',
        'first_name': 'Jane',
        'last_name': 'Doe',
        'age': 25,
        'account_type': 'standard_user'
    }
)
```

Item is a map of attribute name-value pairs, one for each attribute. Only the primary key attributes are required. You can optionally provide other attribute name-value pairs for the item.

users Table: After

username (Partition key)	last_name (Sort key)	first_name	Age	account_type
janedoe	Doe	Jane	25	standard_user

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Updating an item

Update_Item:
Edits an existing item's attributes, or adds a new item to the table if it doesn't already exist

users Table: Before

username (Partition key)	last_name (Sort key)	first_name	Age	account_type
janedoe	Doe	Jane	25	standard_user

```
table.update_item(
    Key = {
        'username': 'janedoe',
        'last_name': 'Doe'
    },
    UpdateExpression = 'SET age = :val1',
    ExpressionAttributeValues = {
        ':val1': 26
    }
)
```

Key is the primary key of the item to be updated.

UpdateExpression defines one or more attributes to be updated.

ExpressionAttributeValues is one or more values that can be substituted in an expression.

users Table: After

username (Partition Key)	last_name (Sort key)	first_name	Age	account_type
janedoe	Doe	Jane	26	standard_user

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Deleting an item

Delete_Item:
Deletes a single item in a table by primary key

users Table: Before

username (Partition key)	last_name (Sort key)	first_name	Age	account_type
janedoe	Doe	Jane	26	standard_user

```
table.delete_item(
    Key = {
        'username': 'janedoe',
        'last_name': 'Doe'
    }
)
```

Key represents the primary key of the item to delete.

users Table: After

username (Partition key)	last_name (Sort key)

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Condition expressions

AccountStatus Table

userId (primary key)	lastFailedLoginTime	accountLocked
1	2021-06-06T19:20+01:00	Y

UpdateItem Operation

Primary key: userId = 1
Set: accountLocked = N
ConditionExpression:
currentLoginTime > lastFailedLoginTime + 24 hours

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Reading an item

Get_Item:
Reads an item from a table

users Table

username (Partition key)	last_name (Sort key)	first_name	Age	account_type
janedoe	Doe	Jane	25	standard_user

Key represents the primary key of the item to retrieve.

```
response = table.get_item (
    Key = {
        'username': 'janedoe',
        'last_name': 'Doe'
    }
)
item = response['Item']
print (item)
```

Expected output

```
{u'username': u'janedoe',
 u'first_name': u'Jane',
 u'last_name': u'Doe',
 u'account_type': u'standard_user',
 u'age': Decimal('25')
}
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Querying a table

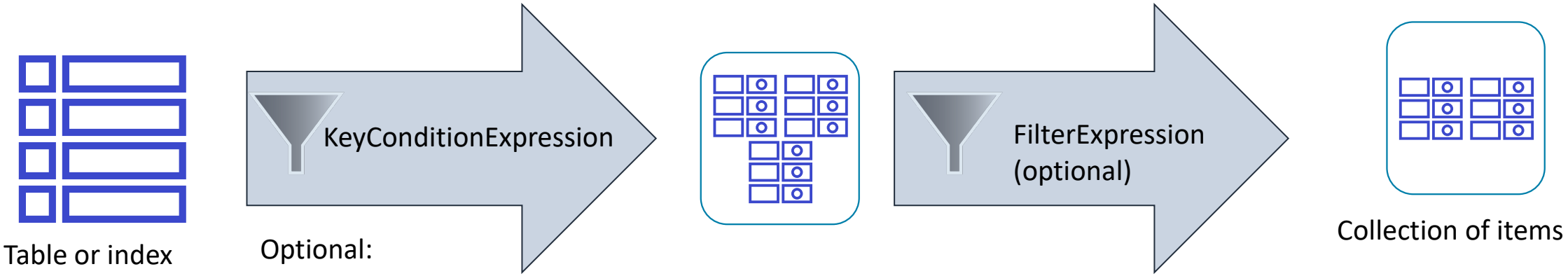


Table or index

Optional:

- ProjectionExpression
- ConsistentRead

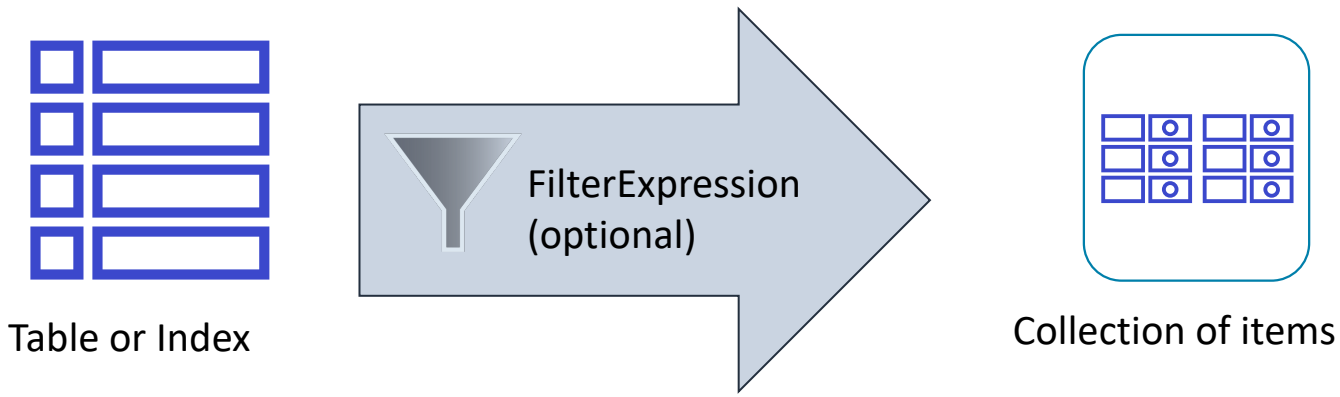
Collection of items

```
response = table.query (
    KeyConditionExpression = Key('username').eq('johndoe')
)
```

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Scanning a table



A Scan operation reads ALL items from the table or index.

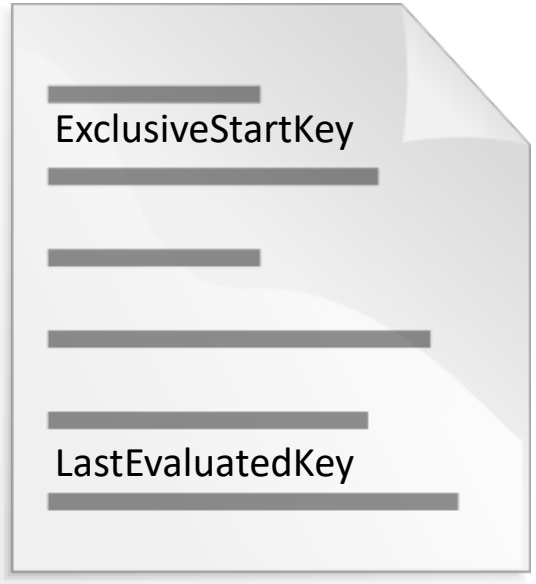
A Query operation is more efficient than a scan operation.

```
response = table.scan (
    FilterExpression = Attr('age').lt(27)
)
```

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Optimizing performance and cost: Limiting returned data

- The 1-MB pagination limit reduces the amount of data that Query or Scan operations return.



- The Limit parameter specifies the maximum number of items that a Query or Scan operation returns.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Batch operations

- BatchGetItem: Read up to 16 MB of data that consists of up to 100 items from multiple tables.
- BatchWriteItem: Write up to 16 MB of data that consists of up to 25 PUT or DELETE requests to multiple tables.
- If one request in a batch fails, the entire operation does not fail.
  - Retry with failed keys and data that is returned by the operation.

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Transactional

- TransactWriteItems
  - Contains a write set
  - Includes one or more PutItem, UpdateItem, and DeleteItem operations across multiple tables
- TransactGetItems
  - Contains a read set
  - Includes one or more GetItem operations across multiple tables
- If one operation fails, the entire transaction fails.

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 9 key takeaways

- DynamoDB has four basic operations:
  - Control: Create and manage tables
  - Data: Create, read, update, delete data
  - Batch: Read and write multiple items
  - Transaction: Coordinated, all-or-nothing change to multiple items

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab 5.1: Working with Amazon DynamoDB

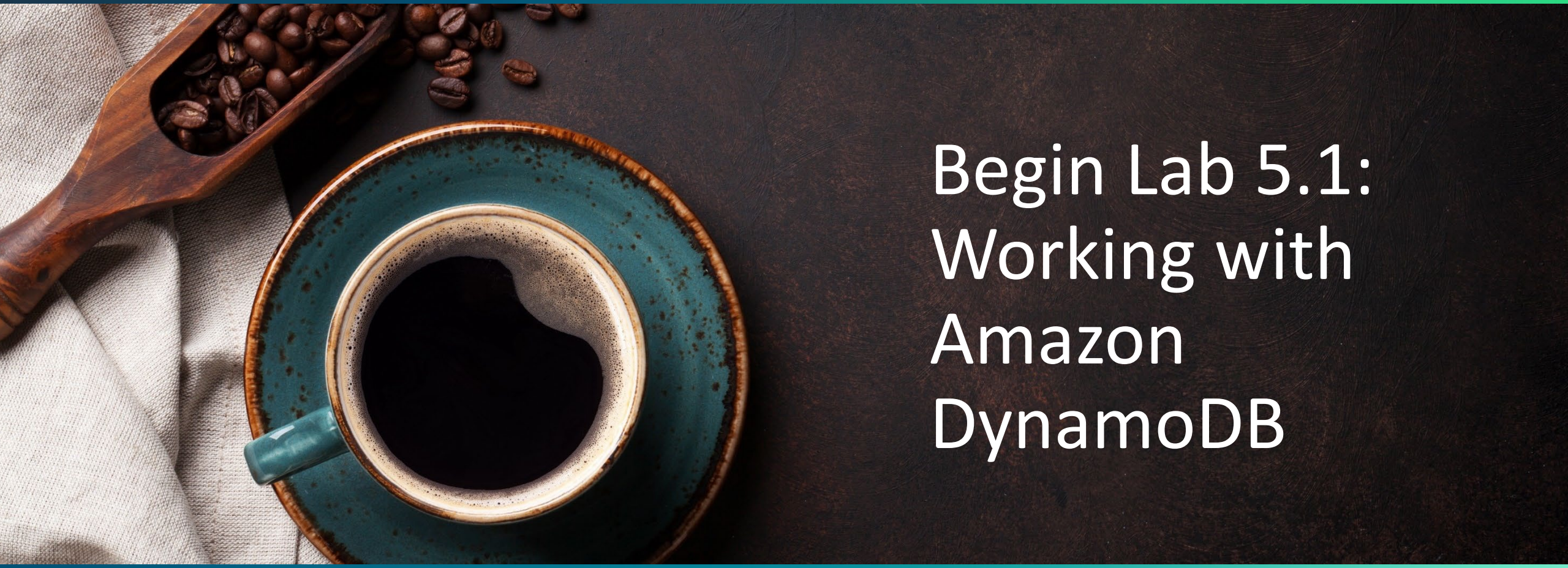
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab: Tasks

1. Preparing the lab
2. Creating a DynamoDB table by using the SDK for Python
3. Working with DynamoDB data – Understanding DynamoDB condition expressions
4. Adding and modifying a single item by using the SDK
5. Adding multiple items by using the SDK and batch processing
6. Querying the table by using the SDK
7. Adding a global secondary index to the table

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

~ 90 minutes



Begin Lab 5.1: Working with Amazon DynamoDB

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab debrief: Key takeaways

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module wrap-up

Module 5: Developing Flexible NoSQL Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module summary

In summary, in this module, you learned how to do the following:

- Recognize different database options on AWS
- Identify the features of DynamoDB
- Describe the components of DynamoDB
- Explain how DynamoDB uses partitions
- Indicate how indexes are used with DynamoDB
- Describe how DynamoDB keeps data consistent
- Recognize when streaming and global tables are used
- Explain the backup and restore process
- Develop with DynamoDB by using the AWS SDKs

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Complete the knowledge check

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Sample exam question

A developer is creating a solution that will use an Amazon DynamoDB table. Anytime the table is updated, the changes must be sent to AWS Lambda. Lambda will then send a message to the Operations team when certain values exist.

How could the developer modify the table to create this solution?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- **Amazon DynamoDB**
- **Updated, the changes must be sent**

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Sample exam question: Response choices

A developer is creating a solution that will use an **Amazon DynamoDB** table. Anytime the table is **updated, the changes must be sent** to AWS Lambda. Lambda will then send a message to the Operations team when certain values exist.

How could the developer modify the table to create this solution?

Choice	Response
A	Create a global secondary index to speed up queries so the developer's code can get faster results.
B	Create a local secondary index to speed up queries so the developer's code can get faster results.
C	Enable DynamoDB Streams on the table.
D	None of the above

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Sample exam question: Answer

The correct answer is C.

Choice	Response
A	Create a global secondary index to speed up queries so the developer's code can get faster results.
B	Create a local secondary index to speed up queries so the developer's code can get faster results.
C	**Enable DynamoDB Streams on the table.**
D	None of the above

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Thank you

Corrections, feedback, or other questions?
Contact us at https://support.aws.amazon.com/#/contacts/aws-academy.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.